Flutter Practice Questions

Exam-style questions (like the "Calculator UI with GridView" question)
with fully line-by-line commented solutions

GridView · ListView · Forms · Cards

How to Use This Sheet

These are written in the same style as your practical exam: a short problem statement, followed by a complete working solution. Almost **every single line** has a comment explaining what it does — read the comments, not just the code, so you understand *why* each line exists (this is what markers usually ask about verbally).

> Paste the full code of any question into `lib/main.dart`, save, and run `flutter run` to see it working before your exam.

1 Calculator Button Layout using GridView

> **Question:** Design a calculator UI. The digits and operators (0-9, +, -, *, /, =, C) should be displayed as buttons arranged in a grid using `GridView`. A display area on top should show the numbers pressed so far.

Why GridView?

`GridView` arranges widgets in a 2D grid (rows AND columns), unlike `Column` (only vertical) or `Row` (only horizontal). `GridView.count()` is the easiest version — you just say how many columns you want (`crossAxisCount`) and it auto-arranges the children into a grid.

```
// import the material design widget library
import 'package:flutter/material.dart';

// entry point of the app - Flutter starts executing from here
void main() {
  runApp(MyApp()); // runApp() attaches the given widget to the screen
}

// root widget of the app, does not change so it is Stateless
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // MaterialApp gives us Material Design styling, themes, navigation etc.
    return MaterialApp(
      home: CalculatorScreen(), // set our calculator screen as the home page
    );
  }
}

// this screen needs to change (display text updates when buttons are pressed)
// so it must be a StatefulWidget
class CalculatorScreen extends StatefulWidget {
  @override
  _CalculatorScreenState createState() => _CalculatorScreenState();
  // createState links this widget to its "brain" (the State class below)
}

// this class actually holds the data and UI logic
class _CalculatorScreenState extends State {

  String display = "0"; // variable holding what is shown on the calculator screen

  // list of all button labels we want to show in the grid, in order
  List buttons = [
    "7", "8", "9", "/",
    "4", "5", "6", "*",
    "1", "2", "3", "-",
    "C", "0", "=", "+",
  ];

  // this function runs every time ANY button is tapped
  void onButtonPressed(String value) {
    setState(() {
      // setState() tells Flutter "data changed, please redraw the UI"
      if (value == "C") {
        display = "0"; // Clear button resets the display
      } else if (value == "=") {
        display = display; // (real calculation logic would go here)
      } else {
        // if display currently shows just "0", replace it instead of appending
        display = (display == "0") ? value : display + value;
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    // Scaffold gives the basic screen structure: app bar + body
    return Scaffold(
      appBar: AppBar(
        title: Text("Simple Calculator"), // title shown on the top bar
      ),
      body: Column( // arrange display area and button grid vertically
        children: [
          // ---------- DISPLAY AREA ----------
          Container(
            width: double.infinity,       // take up full width of screen
            padding: EdgeInsets.all(24),  // inner spacing
            alignment: Alignment.centerRight, // align text to the right like a real calculator
            child: Text(
              display, // shows the current value stored in our 'display' variable
              style: TextStyle(fontSize: 40, fontWeight: FontWeight.bold),
            ),
          ),

          // ---------- BUTTON GRID ----------
          Expanded( // makes the GridView take up all remaining vertical space
            child: GridView.count(
              crossAxisCount: 4, // 4 buttons per row (4 columns)
              children: buttons.map((btnText) {
                // .map() loops through every string in 'buttons' list
                // and turns each one into a button widget
```

```
              return Padding(
                padding: const EdgeInsets.all(4.0), // small gap around each button
                child: ElevatedButton(
                  onPressed: () => onButtonPressed(btnText),
                  // calls onButtonPressed() passing this button's own label

                  child: Text(
                    btnText, // shows the button's number/symbol
                    style: TextStyle(fontSize: 22),
                  ),
                ),
              );
            }).toList(), // .map() returns an Iterable, GridView needs a List, so convert it
          ),
        ),
      ],
    ),
  );
 }
}
```

Exam explanation tip: If asked "why GridView.count and not GridView.builder?" → `GridView.count` is used when you already know the fixed list of items (like calculator buttons, always 16 of them). `GridView.builder` is used when the list is long/dynamic (e.g. loaded from internet or database) because it builds items only as they scroll into view (more efficient).

2 To-Do List using ListView.builder

> **Question:** Create a To-Do list app. The user types a task in a TextField and taps a button to add it to a list. Display all tasks using `ListView.builder`.

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(home: TodoScreen())); // start app directly with TodoScreen
}

// list changes as user adds tasks -> needs to be Stateful
class TodoScreen extends StatefulWidget {
  @override
  _TodoScreenState createState() => _TodoScreenState();
}

class _TodoScreenState extends State {
  // controller lets us read whatever the user types in the TextField
  TextEditingController taskController = TextEditingController();

  // list that stores all the tasks added so far
  List tasks = [];

  // function to add a new task to the list
  void addTask() {
    setState(() {
      // only add if the text field is not empty
      if (taskController.text.isNotEmpty) {
        tasks.add(taskController.text); // add typed text into the list
        taskController.clear();          // clear the text field after adding
      }
    });
  }

  // function to remove a task when its delete icon is tapped
  void removeTask(int index) {
    setState(() {
      tasks.removeAt(index); // remove the task at this position in the list
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("To-Do List")), // top title bar
      body: Padding(
        padding: EdgeInsets.all(12),
        child: Column(
          children: [
            Row( // input field and add button side by side
              children: [
                Expanded( // TextField takes all available horizontal space
                  child: TextField(
                    controller: taskController, // links this field to our controller
                    decoration: InputDecoration(
                      hintText: "Enter a task", // placeholder text
                      border: OutlineInputBorder(), // draws a box border
                    ),
                  ),
                ),
                IconButton(
                  icon: Icon(Icons.add_circle, size: 36, color: Colors.blue),
                  onPressed: addTask, // calls addTask() when tapped
                ),
              ],
            ),
            SizedBox(height: 12), // gap below the input row

            // Expanded lets ListView fill the rest of the screen height
            Expanded(
              child: ListView.builder(
                itemCount: tasks.length, // total number of items to build
                itemBuilder: (context, index) {
                  // this function runs once per item in the list
                  return Card( // Card gives each task a nice elevated box look
                    child: ListTile(
                      title: Text(tasks[index]), // show the task text
                      trailing: IconButton( // delete icon on the right
                        icon: Icon(Icons.delete, color: Colors.red),
                        onPressed: () => removeTask(index), // remove this specific task
                      ),
                    ),
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

3 Login Form UI

Question: Design a simple login screen with an email field, a password field (hidden text), and a login button that shows a message when pressed.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(home: LoginScreen()));
}

class LoginScreen extends StatefulWidget {
  @override
  _LoginScreenState createState() => _LoginScreenState();
}

class _LoginScreenState extends State {
  // controllers to read email and password typed by the user
  TextEditingController emailController = TextEditingController();
  TextEditingController passwordController = TextEditingController();

  void login() {
    // showDialog pops up a small message box on screen
    showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title: Text("Login Attempt"),
        // shows whatever was typed in the email field
        content: Text("Email entered: ${emailController.text}"),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context), // closes the dialog
            child: Text("OK"),
          ),
        ],
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Login")),
      body: Padding(
        padding: EdgeInsets.all(20), // spacing around the whole form
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center, // center everything vertically
          crossAxisAlignment: CrossAxisAlignment.stretch, // fields stretch full width
          children: [
            Text(
              "Welcome Back",
              style: TextStyle(fontSize: 26, fontWeight: FontWeight.bold),
              textAlign: TextAlign.center,
            ),
            SizedBox(height: 30),

            // email input field
            TextField(
              controller: emailController,
              decoration: InputDecoration(
                labelText: "Email",
                prefixIcon: Icon(Icons.email), // small icon inside the field
                border: OutlineInputBorder(),
              ),
            ),
            SizedBox(height: 16),

            // password input field
            TextField(
              controller: passwordController,
              obscureText: true, // hides typed characters with dots, for passwords
              decoration: InputDecoration(
                labelText: "Password",
                prefixIcon: Icon(Icons.lock),
                border: OutlineInputBorder(),
              ),
            ),
            SizedBox(height: 24),

            // login button
            ElevatedButton(
              onPressed: login, // runs the login() function above
              style: ElevatedButton.styleFrom(padding: EdgeInsets.all(14)),
              child: Text("LOGIN", style: TextStyle(fontSize: 16)),
            ),
          ],
        ),
      ),
    );
  }
}
```

4 Profile Card using Row, Column & CircleAvatar

> **Question:** Create a profile card UI showing a circular profile picture, name, designation, and a row of stats (Posts, Followers, Following).

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(home: ProfileScreen()));
}

// this screen never changes -> StatelessWidget is enough
class ProfileScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Profile")),
      body: Center(
        child: Card( // Card widget = a box with rounded corners + shadow
          margin: EdgeInsets.all(20),
          elevation: 6, // controls how strong the shadow looks
          child: Padding(
            padding: EdgeInsets.all(20),
            child: Column( // stack all profile info vertically
              mainAxisSize: MainAxisSize.min, // card only as tall as its content
              children: [
                // CircleAvatar shows a circular image, common for profile pics
                CircleAvatar(
                  radius: 50, // controls the size of the circle
                  backgroundImage: NetworkImage(
                    "https://i.pravatar.cc/150", // sample placeholder image URL
                  ),
                ),
                SizedBox(height: 12),

                Text("Ananya Sharma",
                    style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
                Text("Flutter Developer",
                    style: TextStyle(fontSize: 14, color: Colors.grey)),
                SizedBox(height: 16),

                // row of 3 stats spread evenly across the width
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                  children: [
                    buildStat("120", "Posts"),     // helper function call, defined below
                    buildStat("3.2k", "Followers"),
                    buildStat("180", "Following"),
                  ],
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }

  // helper function that builds one stat column (number above label)
  // keeping this separate avoids repeating the same code 3 times
  Widget buildStat(String number, String label) {
    return Column(
      children: [
        Text(number, style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
        Text(label, style: TextStyle(fontSize: 12, color: Colors.grey)),
      ],
    );
  }
}
```

5 Image Gallery using GridView.builder

Question: Display a scrollable photo gallery grid with 3 images per row using `GridView.builder`.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(home: GalleryScreen()));
}

class GalleryScreen extends StatelessWidget {
  // list of sample image URLs to display in the gallery
  final List imageUrls = List.generate(
    12, // generate 12 items
    (index) => "https://picsum.photos/id/${index + 10}/200", // different picture each time
  );

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Photo Gallery")),
      // GridView.builder is used here because the list could be long/dynamic
      body: GridView.builder(
        padding: EdgeInsets.all(8), // spacing around the whole grid
        gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount: 3,      // 3 images per row
          crossAxisSpacing: 6,    // horizontal gap between images
          mainAxisSpacing: 6,     // vertical gap between images
        ),
        itemCount: imageUrls.length, // total number of images to build
        itemBuilder: (context, index) {
          // runs once for every image, 'index' tells which one
          return ClipRRect( // clips the image into rounded corners
            borderRadius: BorderRadius.circular(8),
            child: Image.network(
              imageUrls[index], // load this specific image's URL
              fit: BoxFit.cover, // scale image to fill its box neatly
            ),
          );
        },
      ),
    );
  }
}
```

Extra: GridView Quick Reference

```
// OPTION 1: Fixed, known number of items
GridView.count(
  crossAxisCount: 2,      // number of columns
  children: [ Widget1, Widget2, Widget3 ], // you provide widgets directly
)

// OPTION 2: Dynamic / long list of items (more efficient, builds on demand)
GridView.builder(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 2),
  itemCount: myList.length,          // how many items total
  itemBuilder: (context, index) {
    return Text(myList[index]);      // what each grid cell should look like
  },
)
```

Exam pattern to remember: Almost any "design a UI for ___" question follows: `Scaffold → AppBar → body → (Column/Row/GridView/ListView) → individual widgets`. If the list of things to show is fixed and small, use `Column` / `Row` / `GridView.count`. If it's a scrollable list of many similar items, use `ListView.builder` / `GridView.builder`.